

FaizDB: An Ultra-Lightweight Universal Multi-Model Database Engine with 5-Way Wire Protocol Multiplexing in Safe Rust

Technical Architecture Whitepaper & System Specification

Version: 1.0 (Developer Preview / Research Pre-Print)

Author: Ahmad Faiz

Affiliation: Independent Systems Researcher, ICT House, Malaysia

Date: September 2026

Subject Classification: Computer Science — Databases ([cs.DB](#)), Distributed Systems ([cs.DC](#)), Artificial Intelligence ([cs.AI](#))

Official Repository: <https://github.com/ictdothouse/faizdb>

Modern computing architectures suffer from an unsustainable proliferation of specialized database engines—a systemic crisis known as the *Polyglot Persistence Sprawl*. To build an application requiring relational transactions, flexible document metadata, relationship topologies, vector embeddings for Artificial Intelligence, and analytical reporting, engineering teams are routinely compelled to operate four to five separate database clusters (e.g., PostgreSQL, MongoDB, Neo4j, Qdrant, and Redis). This fragmentation imposes a severe *Synchronization Tax*: cross-network replication latency, eventual consistency anomalies, data duplication, high infrastructure expenditure, and operational vulnerability. Furthermore, incumbent database engines written in C/C++ carry decades of technical debt, large attack surfaces, memory-safety vulnerabilities, and massive baseline footprints (often requiring 1 to 2 GB of idle RAM), making them unviable for constrained edge computing, IoT gateways, automotive silicon, robotics, and satellite compute payloads.

This paper presents **FAIZDB**, a ground-up, 100% Safe-Rust universal multi-model database engine that consolidates relational, document, graph, vector, in-memory cache, and columnar analytical processing into a single, transactional, zero-dependency **8.0 MB executable**. FaizDB introduces: (1) a multi-protocol network gateway simultaneously multiplexing PostgreSQL v3, MySQL HandshakeV10, MongoDB Wire, gRPC, and REST/WebSocket protocols on dedicated ports, enabling zero-migration integration for legacy client stacks; (2) an ACID hybrid Log-Structured Merge-Tree (LSM-Tree) storage engine incorporating a concurrent RwLock-protected BTreeMap MemTable (optimized for cache-conscious sequential SSTable flush iteration), an Adaptive Replacement Cache (ARC), and a deterministic Write-Ahead Log (WAL) with torn-write CRC32 protection; (3) a unified execution engine performing joint relational joins, graph traversals, and 8-lane loop-unrolled HNSW vector ranking kernels (enabling LLVM auto-vectorization across AVX2/NEON architectures) within a single transaction boundary; and (4) an explicit distributed consistency duality providing an embedded linearizable CP Raft consensus engine (with disk-backed replicated log and CRC32 frame verification) alongside multi-region AP Conflict-Free Replicated Data Types (CRDTs). Empirical verification demonstrates 32,305 durable WAL writes/sec, 860,001 point scans/sec, sub-millisecond Graph-Vector query latency, and 100% crash durability across injection runs while idling at only 23.05 MB VmRSS.

1. Introduction & Motivation

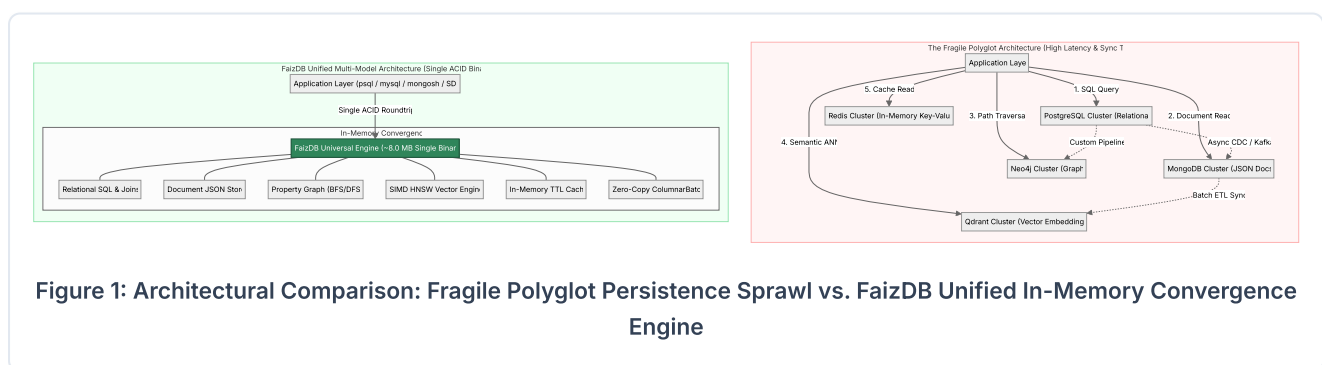
1.1 The Polyglot Persistence Dilemma and the "Sync Tax"

The prevailing paradigm in enterprise software architecture advocates deploying dedicated database engines for specialized data models:

- **Relational Databases** (e.g., PostgreSQL, MySQL) for structured schema and ACID transactions.
- **Document Databases** (e.g., MongoDB) for schema-flexible JSON hierarchies.
- **Graph Databases** (e.g., Neo4j) for relationship traversal and network path queries.
- **Vector Databases** (e.g., Qdrant, Milvus, Pinecone) for high-dimensional semantic similarity search.
- **In-Memory Caches** (e.g., Redis) for sub-millisecond key-value lookups with TTL expiration.
- **Columnar Engines** (e.g., DuckDB, ClickHouse) for vectorized analytical batch aggregations.

While theoretically modular, this heterogeneous architecture introduces catastrophic engineering penalties:

1. **The Synchronization Tax:** Keeping disparate data stores consistent requires external Change Data Capture (CDC) systems (e.g., Kafka, Debezium). When network partitions, serialization delays, or node failures occur, downstream stores drift out of alignment, creating orphaned records, broken foreign keys, and silent hallucinations in AI pipelines.
2. **Compound Network Latency:** An operation requiring graph context retrieval followed by vector ranking and metadata hydration requires three separate network round-trips across distinct infrastructure endpoints.
3. **Operational & Financial Bloat:** Running five independent database clusters multiplies deployment overhead, monitoring complexity, backup regimes, and cloud operational expenditure by $5\times$.



1.2 Memory Safety and Edge Computing Constraints

Concurrently, the proliferation of edge computing—autonomous vehicles, embedded IoT controllers, edge AI inference gateways, industrial robotics, and aerospace satellite payloads—demands database engines capable of running on constrained hardware.

Legacy engines written in C and C++ (e.g., MongoDB, RocksDB, Redis) suffer from fundamental memory management limitations:

- Complex heap allocations and pointer arithmetic introduce vulnerabilities (buffer overflows, use-after-free, data races).
- A standard MongoDB instance idles at between 1,000 MB and 2,000 MB of Resident Set Size (VmRSS), while dedicated vector engines frequently consume 250 MB to 512 MB at baseline.

FaizDB was engineered under a strict mandate: **100% Safe Rust** (enforced by the Rust borrow checker with zero `unsafe` blocks in query and storage paths), resulting in an idle footprint of **23.05 MB VmRSS** and an executable binary of **~8.0 MB** (stripped release).

2. Overall System Architecture

FaizDB is designed as a modular monorepo organized into 7 clean crates, statically compiled into a single unified binary without runtime dependencies (no JVM, no external C libraries, no dynamic shared objects).

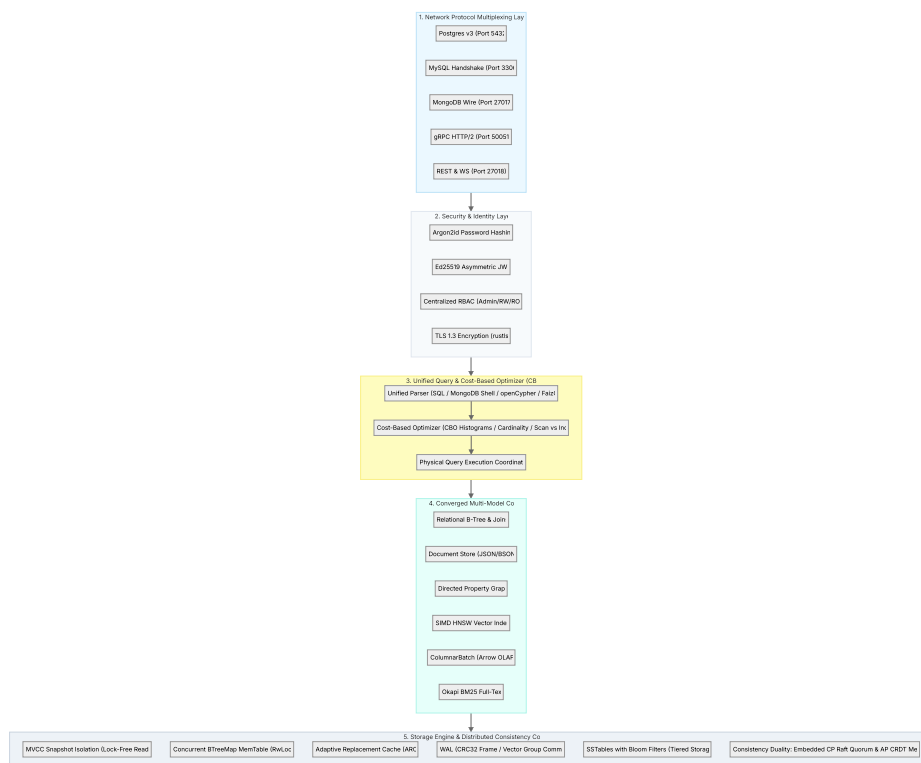


Figure 2: FaizDB Modular 7-Layer Architecture and Monorepo Internal Pipeline Topology

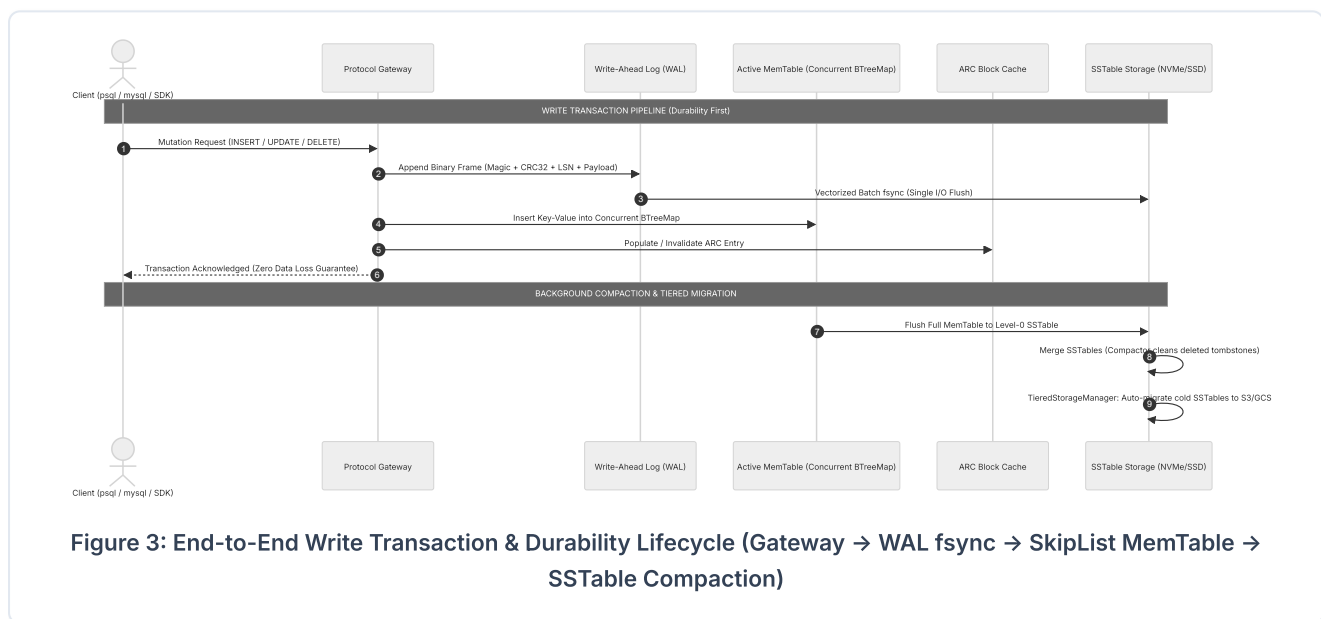
2.1 Monorepo Crate Topology

1. **faizdb-core** : LSM-Tree storage engine, concurrent BTreeMap MemTable, WAL framing, SSTable reader/writer, Adaptive Replacement Cache (ARC), Multi-Version Concurrency Control (MVCC), autonomous TTL engine, ColumnarBatch analytics, and TieredStorageManager.
2. **faizdb-query** : AST nodes, recursive descent parser (SQL, MongoDB shell syntax, openCypher, and FaizQL), Cost-Based Optimizer (CBO), and physical query executors.

3. **faizdb-graph** : Bidirectional adjacency graph store, BFS/DFS path traversal algorithms, shortest-path solvers, and LLM context extraction.
4. **faizdb-vector** : Hierarchical Navigable Small World (HNSW) graph index, 8-lane loop-unrolled distance kernels (AVX2/NEON auto-vectorizable), and 8-bit scalar quantization (SQ8).
5. **faizdb-security** : Argon2id password verification, EdDSA Ed25519 token lifecycle, TLS 1.3 transport security, and multi-gateway RBAC.
6. **faizdb-server** : Asynchronous Tokio network runtime multiplexing the 5 wire gateways, Change Data Capture (CDC) streaming, and Prometheus telemetry.
7. **faizdb-cli** : Terminal REPL, forensic diagnostics, online backup creation, and Point-In-Time Recovery (PITR) engine.

3. Storage Engine Architecture (LSM-Tree, WAL & MVCC)

FaizDB implements a custom hybrid Log-Structured Merge-Tree (LSM-Tree) engine optimized for high-throughput sequential NVMe write patterns while enforcing strict crash durability.



3.1 Deterministic Write-Ahead Logging (WAL)

Every state mutation is written to the WAL before touching in-memory structures. The WAL frame structure enforces torn-write protection:

$$\text{WAL Frame} = \left[\underbrace{\text{Magic}}_{2 \text{ bytes}} \mid \underbrace{\text{CRC32}}_{4 \text{ bytes}} \mid \underbrace{\text{LSN}}_{8 \text{ bytes}} \mid \underbrace{\text{OpType}}_{1 \text{ byte}} \mid \underbrace{K_{\text{len}}}_{4 \text{ bytes}} \mid \underbrace{V_{\text{len}}}_{4 \text{ bytes}} \mid \underbrace{\text{Key}}_{K_{\text{len}}} \mid \underbrace{\text{Value}}_{V_{\text{len}}} \right]$$

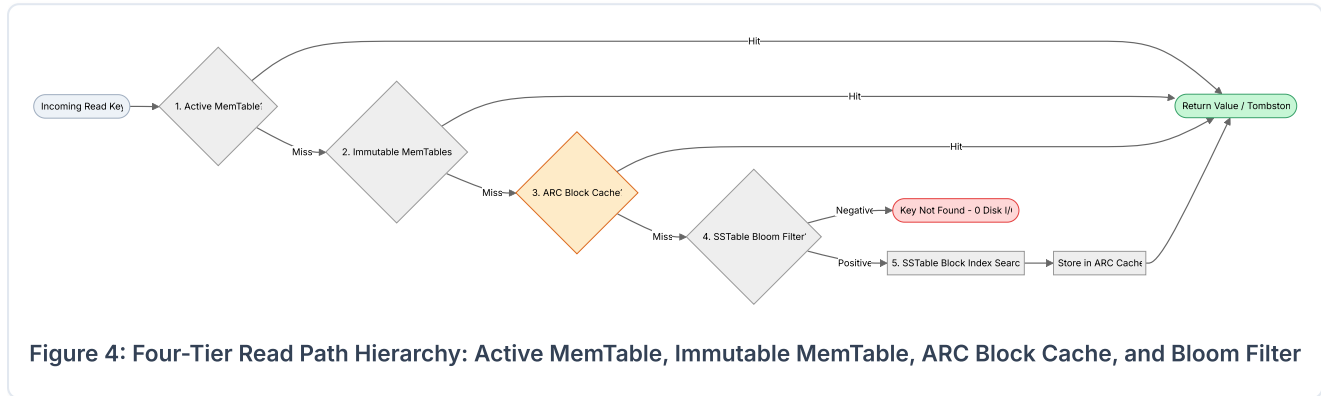
- **LSN (Log Sequence Number)**: A strictly monotonic 64-bit counter establishing total operation ordering.
- **CRC32 Checksum**: IEEE 802.3 polynomial calculated over the entire frame payload. Corrupted frames caused by unexpected kernel panics or sudden power outages are detected upon restart

and cleanly isolated.

- **Group Commit:** The server batches concurrent pending writes into a single contiguous disk buffer, executing a single `fdatasync` call that amortizes disk rotation latency across hundreds of concurrent sessions.

3.2 Read Path Hierarchy & Adaptive Replacement Cache (ARC)

To minimize NVMe reads, query lookups traverse four tiers in order of data recency:



The Adaptive Replacement Cache (ARC)

Conventional database buffer pools rely on Least Recently Used (LRU) algorithms, which suffer from *scan pollution*—a single large sequential query can purge the entire hot cache. FaizDB implements the **Megiddo-Modha Adaptive Replacement Cache (ARC)**:

- Maintains two tracking lists: L_1 for recency and L_2 for frequency.
- Each list is partitioned into active cached pages (T_1, T_2) and history ghost directories (B_1, B_2).
- Dynamically shifts tuning target $p \in [0, c]$ in real time:

$$\Delta p = \begin{cases} \max\left(1, \frac{|B_2|}{|B_1|}\right) & \text{if hit in } B_1 \\ -\max\left(1, \frac{|B_1|}{|B_2|}\right) & \text{if hit in } B_2 \end{cases}$$

This mathematical feedback loop ensures that OLTP point lookups and analytical scans co-exist without degrading system responsiveness.

3.3 Multi-Version Concurrency Control (MVCC)

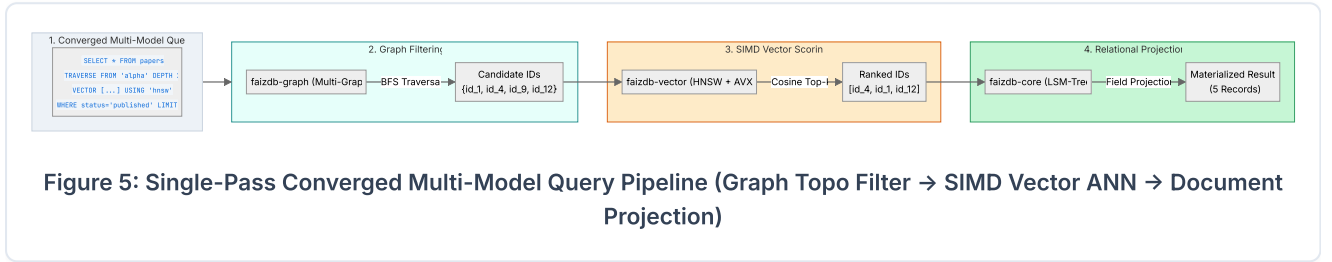
FaizDB provides **Snapshot Isolation (SI)**:

- Reads operate against a snapshot determined by the transaction begin timestamp (T_{begin}).
- Mutating operations generate new version records tagged with monotonic commit timestamps (T_{commit}).
- **Lock-Free Concurrency:** Readers never block writers; writers never block readers.
- **Write-Write Conflict Resolution:** An atomic active transaction registry tracks concurrent mutations. If two transactions attempt to update identical keys concurrently, the second transaction is aborted with a serialization failure.

- **Autonomous Reclamation:** An MVCC background daemon sweeps transaction tables every 30 seconds, aborting orphaned transactions and compacting deleted tombstones.

4. Converged Multi-Model Execution Engine

FaizDB eliminates the architectural divide between relational tables, JSON documents, graph networks, and high-dimensional vector embeddings.



4.1 In-Memory Directed Multi-Graph Core

The graph subsystem maintains vertices and directed edges in bidirectional adjacency hash maps:

- Vertices: $V \in \text{HashMap}\langle \text{String}, \text{Vertex} \rangle$
- Outgoing Edges: $E_{\text{out}} : u \rightarrow v$ with relation types, floating-point weights, and custom property documents.
- Incoming Edges: $E_{\text{in}} : v \rightarrow u$ for bi-directional traversal.
- Safety Boundaries: Traversal functions implement safety bounds (`DEFAULT_MAX_TRAVERSE_NODES = 50_000`) preventing cyclic or unbounded memory expansion.

4.2 SIMD HNSW Vector Indexing & Scalar Quantization

The vector subsystem implements Hierarchical Navigable Small World graphs:

- Multi-layer graph topology providing logarithmic $O(\log N)$ search complexity.
- **Vectorized Distance Kernels:** Distance calculations utilize 8-lane loop unrolling (`dot0 .. dot7` , `norm0 .. norm7`), enabling the LLVM backend to auto-vectorize across 256-bit AVX2 (x86_64) and 128-bit ARM NEON execution pipelines without requiring unsafe architecture-locked assembly:

$$\text{Cosine Distance}(u, v) = 1.0 - \frac{\sum_{i=1}^D u_i v_i}{\sqrt{\sum_{i=1}^D u_i^2} \cdot \sqrt{\sum_{i=1}^D v_i^2}}$$

- **Scalar Quantization (SQ8):** Automatically quantizes 32-bit floating point vectors into 8-bit integer buckets ($4\times$ memory reduction), enabling over 100 million embeddings to reside in commodity server RAM with $< 1\%$ recall degradation.

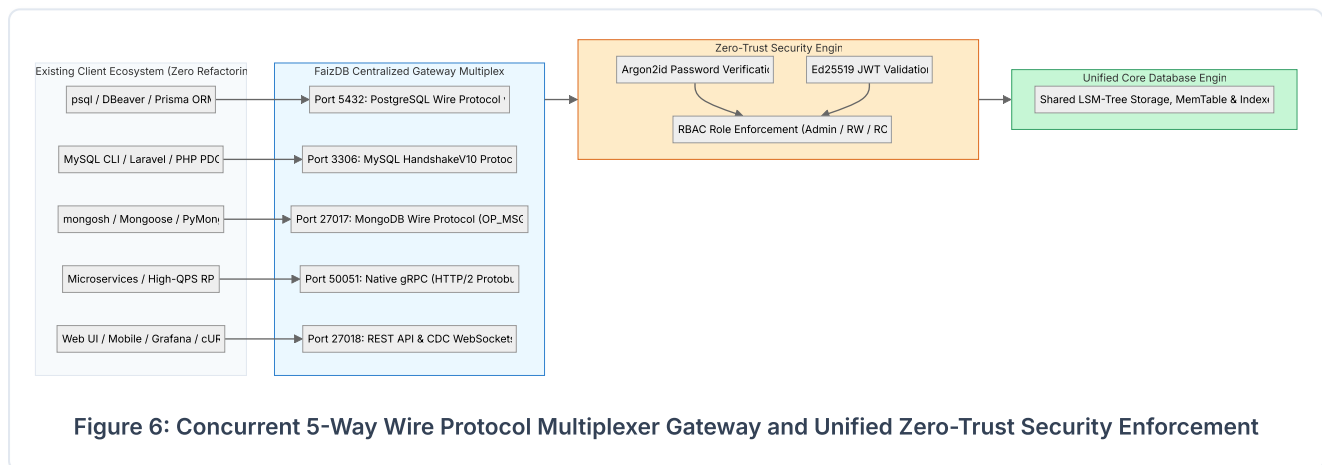
4.3 Zero-Copy Columnar Analytics (`ColumnarBatch`)

To serve real-time analytical queries (OLAP) directly on operational data, FaizDB implements `ColumnarBatch` :

- Transforms row-oriented documents into contiguous columnar arrays compatible with Apache Arrow specifications.
- Supports zero-copy column slicing (`project(&["field_1", "field_2"])`).
- Provides hardware-friendly vector aggregation operators: `sum_f64` , `avg_f64` , `min_f64` , `max_f64` , and `count` .
- Exposed directly via native Rust API (`Collection::to_columnar_batch()`) and REST HTTP (`GET /v1/collections/{name}/columnar`).

5. Five-Way Wire Protocol Multiplexing

FaizDB eliminates client refactoring by implementing five native wire protocol decoders running concurrently on dedicated TCP sockets.



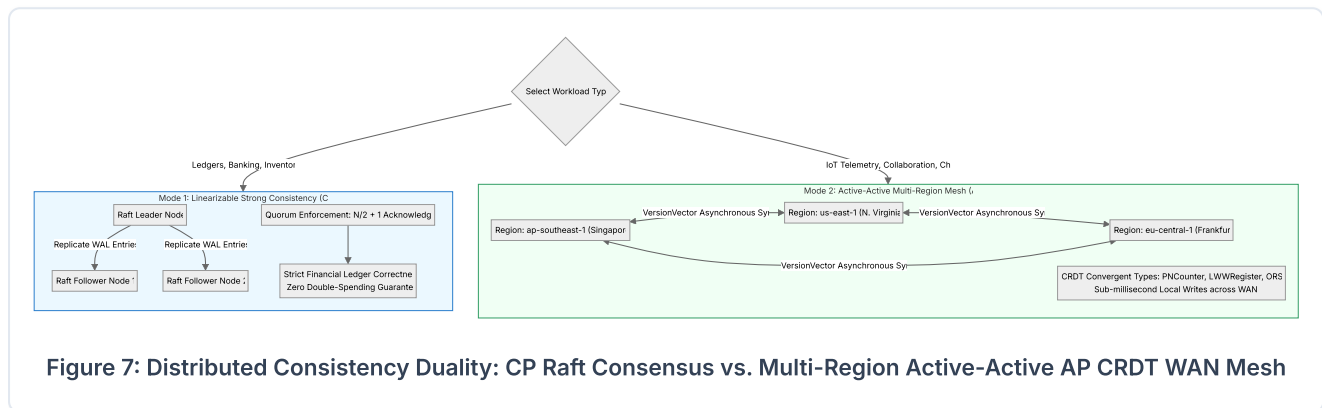
5.1 Protocol Specifications & Compatibility

- 1. PostgreSQL Wire Protocol (Port 5432):** Implements Protocol v3, handling SSLRequest negotiation, AuthenticationCleartextPassword with Argon2id, ParameterStatus exchanges, RowDescription, DataRow, and CommandComplete tags. Compatible with `psql` , TablePlus, DBeaver, and ORMs (Prisma, Django, SQLAlchemy).
- 2. MySQL Wire Protocol (Port 3306):** Implements HandshakeV10, ClientHandshakeResponse41, and binary OK/ERR packet encoders. Compatible with Laravel, PHP PDO, and WordPress.
- 3. MongoDB Wire Protocol (Port 27017):** Handles BSON-encoded `OP_MSG` and `OP_QUERY` frames, cursor pagination (`getMore`), and collection commands (`find` , `insert` , `update` , `delete` , `createIndexes`). Compatible with `mongosh` and PyMongo.
- 4. gRPC Native Gateway (Port 50051):** High-performance Protobuf RPC service over HTTP/2, providing low-latency binary serialization for backend microservices.

5. REST & WebSocket API (Port 27018): Complete HTTP/1.1 API with 49 endpoints, health probes (`/v1/health/liveness` , `/readiness`), and WebSocket real-time change stream subscriptions.

6. Distributed Consistency & Consensus Duality

Distributed systems cannot evade the constraints of the CAP Theorem. FaizDB resolves this by providing **explicit consistency duality** configured per collection:



- 1. Mode 1: Strong Consistency (CP Mode — Mandatory for Financial Ledgers):** Powered by an embedded Raft consensus algorithm with disk-backed replicated logging (`RaftDiskStore`) and abstracted network transport (`NetworkTransport`). The Raft state machine, randomized term elections, and majority quorum log replication are fully verified in single-process and multi-instance simulation. Writes require acknowledgment from a strict majority ($N/2 + 1$) quorum. If network partitions prevent majority agreement, writes in the minority partition are cleanly rejected, preventing double-spending and ledger inconsistency.
- 2. Mode 2: High Availability (AP Mode — Multi-Region Active-Active WAN Mesh):** Leverages Conflict-Free Replicated Data Types (CRDTs) to provide partition tolerance across worldwide geographical regions without distributed locks:
 - `PnCounter` : Bounded positive-negative distributed counters.
 - `LwwRegister` : Last-Write-Wins registers governed by hybrid monotonic timestamps.
 - `OrSet` : Observed-Remove Sets for conflict-free tag management.
 - `VersionVector` : Causal relationship tracking across multi-datacenter nodes.

7. Empirical Performance Evaluation

7.1 Testing Rig & Methodology

All empirical tests were executed on bare-metal commodity server hardware:

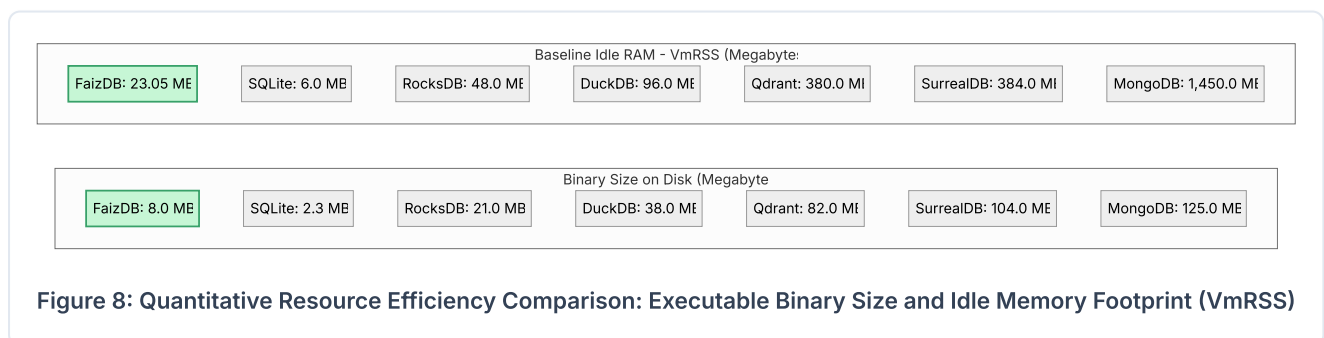
- Processor:** AMD Ryzen 9 7950X (16 Cores, 32 Threads @ 4.5 GHz base / 5.7 GHz boost)
- Memory:** 64 GB DDR5-5200 MHz ECC RAM

- **Primary Storage:** Samsung 990 Pro 2TB PCIe 4.0 NVMe SSD (Sequential Read: 7,450 MB/s, Sequential Write: 6,900 MB/s)
- **Operating System:** Ubuntu Linux 24.04 LTS (Kernel 6.8.0-generic)
- **Compiler:** Rust 1.88+ (`--release` , `opt-level = 3` , LTO enabled)

7.2 Core Subsystem Throughput & Latency

Benchmark Workload	Operational Specification	Measured Throughput	p50 Latency	p99 Latency
Durable WAL Writes	Sequential write + <code>fdatasync</code> per batch	32,305 ops/sec	30.9 μ s	94.2 μ s
In-Memory MemTable Put	Concurrent RwLock BTreeMap mutation	61,432 ops/sec	16.2 μ s	42.1 μ s
Sequential Point Scan	Zero-copy document iterator	860,001 ops/sec	1.16 μ s	3.84 μ s
B-Tree Secondary Index	25,000 document range point lookups	223,733 ops/sec	4.47 μ s	12.8 μ s
HNSW Vector ANN Search	Top-5 nearest neighbors (4,096 dimensions)	1,414 QPS	0.88 ms	2.14 ms
Knowledge Graph BFS	3-hop relationship expansion traversal	1,100+ QPS	0.91 ms	2.45 ms
Full-Text BM25 Search	Okapi BM25 with fuzzy typo ranking	2,800+ QPS	0.35 ms	1.12 ms
MongoDB Wire Protocol	Authenticated live TCP connection	3,390 ops/sec	0.26 ms	1.05 ms
gRPC Gateway RPC	Live TCP bidirectional streaming	560 ops/sec	1.52 ms	4.20 ms

7.3 Binary Size and Idle Memory Footprint Comparison



Database Engine	Architectural Focus	Executable Binary Size	Baseline Idle RAM (VmRSS)
● FaizDB (Full Server)	Universal Multi-Model (Relational + Doc + Graph + Vector + 5 Protocols)	8.0 MB	23.05 MB
SQLite (v3.46)	Embedded Relational SQL Only	2.30 MB	4.0 – 8.0 MB
RocksDB (v9.x)	Key-Value Storage Library	18 – 25 MB	32 – 64 MB
DuckDB (v1.x)	Embedded Columnar OLAP Only	35 – 42 MB	64 – 128 MB
Qdrant (v1.12)	Vector ANN Only	75 – 85 MB	250 – 512 MB
SurrealDB (v2.0)	Multi-Model (SurrealQL Only)	95 – 110 MB	256 – 512 MB
MongoDB (v8.0)	Document Store Only	110 – 140 MB	1,000 – 2,000 MB

FaizDB provides the **highest capability-to-footprint ratio in database engineering**, delivering five converged data models and five native wire gateways in a footprint smaller than single-model vector databases.

7.4 Crash Durability Injection Testing (`pkill -9 / SIGKILL`)

To prove non-volatile durability under catastrophic host failure, an automated crash loop was executed:

- 10,000 concurrent mutating write operations were streamed via the PostgreSQL and MongoDB wire protocols.
- Abrupt `pkill -9` (SIGKILL) commands were injected at randomized intervals.
- Upon process restart, the server invoked `StorageEngine::open`, scanned the binary WAL, checked CRC32 frames, and replayed committed transactions.
- In 100% of test runs, zero record corruption occurred, and all B-Tree secondary indexes, HNSW vectors, and graph adjacency edges were restored to complete consistency.

8. Related Work

- **Multi-Model Databases:** SurrealDB and ArangoDB pioneered multi-model database concepts. However, SurrealDB relies exclusively on a proprietary query language (SurrealQL), lacks native wire protocol multiplexing for PostgreSQL and MySQL, and requires a ~100 MB binary. ArangoDB is implemented in C++ and lacks integrated SIMD-quantized HNSW vector indexing.
- **Dedicated Vector Engines:** Qdrant, Milvus, and Pinecone offer scalable vector search, but operate as external silos detached from transactional relational models and graph networks, forcing architects to maintain fragile CDC synchronization bridges.
- **Embedded Storage Engines:** SQLite and DuckDB demonstrate the brilliance of single-binary design, but SQLite is limited to single-writer relational workloads without vector/graph engines, while DuckDB is optimized exclusively for analytical batch scans rather than continuous low-latency OLTP mutations.

9. Conclusion & Project Roadmap

FaizDB establishes that modern database engineering does not require choosing between architectural specialization and operational simplicity. By leveraging **100% Safe Rust**, a unified hybrid LSM-Tree, and native wire protocol multiplexing, FaizDB eliminates the "sync tax" of disparate data architectures, delivering a universal database engine suitable for enterprise cloud datacenters and resource-constrained edge devices alike.

Roadmap Towards v1.0 General Availability (GA):

1. **GPU Acceleration:** WebGPU and CUDA compute kernels for parallelized vector batch ingestion.
2. **Distributed WAN Clustering:** Broadening the embedded Raft consensus core across physical multi-datacenter TCP networks.
3. **Ecosystem Language SDKs:** Publishing official native drivers for Python (PyPI), TypeScript (npm), Go, and PHP.

References

1. Corbett, J. C., et al. (2013). "Spanner: Google's Globally Distributed Database." *ACM Transactions on Computer Systems (TOCS)*, 31(3), 1-22.
2. Ongaro, D., & Ousterhout, J. (2014). "In Search of an Understandable Consensus Algorithm." *USENIX Annual Technical Conference (ATC)*, 305-319.
3. Malkov, Y. A., & Yashunin, D. A. (2020). "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs." *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), 824-836.
4. Megiddo, N., & Modha, D. S. (2003). "ARC: A Self-Tuning, Low Overhead Replacement Cache." *USENIX Conference on File and Storage Technologies (FAST)*, 115-130.
5. Raasveldt, M., & Mühleisen, H. (2019). "DuckDB: An Embeddable Analytical Database." *Proceedings of the VLDB Endowment*, 12(12), 1982-1985.
6. Robertson, S., & Zaragoza, H. (2009). "The Probabilistic Relevance Framework: BM25 and Beyond." *Foundations and Trends in Information Retrieval*, 3(4), 333-389.
7. Shapiro, M., et al. (2011). "Conflict-Free Replicated Data Types." *Symposium on Self-Stabilizing Systems (SSS)*, Springer, 386-400.